Part C: Bilinear Element Model and Testing Coded with AI

Mesut

DEP 521E – Special Topics in Earthquake Engineering, Homework 2

November 5, 2026

Abstract

This report presents a Python implementation of a rate-independent bilinear hysteretic element, developed with the help of an AI coding assistant as required in Part C of the assignment. The element is described by a primary stiffness k_p, a secondary stiffness k_s and two parallel border lines that bound the force. The code is driven through the displacement history given in the assignment, and the resulting displacement and force time histories and the force–displacement hysteresis are reported. The results are checked against hand calculations and against an independent fine-step path-tracing solution.

1 Problem definition

The bilinear element of Fig. 1 in the assignment is defined by the yield displacement d_y, the yield force f_y and the secondary stiffness k_s. The primary (elastic) stiffness and the characteristic strength (the force intercept of the borders) are

$$k_p = \frac{f_y}{d_y}, \qquad q_y = f_y - k_s d_y. \tag{1}$$

At any displacement x the force is bounded by the *top* and *bottom* borders,

$$F_{\text{top}}(x) = q_y + k_s x, \qquad F_{\text{bot}}(x) = -q_y + k_s x, \tag{2}$$

and the region between them is the *interior region*, in which the element responds with the primary stiffness k_p. On a border, loading away from the interior follows the border with stiffness k_s, while unloading moves back into the interior with stiffness k_p.

The function required by the assignment has the form

```
df, param_a = bilinear(dy, fy, k2, x_ini, f_ini, dx, param_a)
```

where `x_ini` and `f_ini` are the displacement and force at the start of the step, `dx` is the displacement increment and `param_a` carries the state of the element (`interior`, `top` or `bottom`). The function returns the force increment `df` and the updated state.

2 Formulation

2.1 Elastic trial and return to the border

For a step from x_i to $x_{i+1} = x_i + \Delta x$, an elastic trial force is computed with the primary stiffness,

$$F^{\text{trial}} = F_i + k_p\,\Delta x. \tag{3}$$

Because $k_s < k_p$, the trial force can lie outside the borders only if the element has yielded during the step. In that case the step must end on the border that was crossed. Hence

$$F_{i+1} = \begin{cases} F_{\text{top}}(x_{i+1}), & F^{\text{trial}} \geq F_{\text{top}}(x_{i+1}) \quad \text{(state: top)}, \\ F_{\text{bot}}(x_{i+1}), & F^{\text{trial}} \leq F_{\text{bot}}(x_{i+1}) \quad \text{(state: bottom)}, \\ F^{\text{trial}}, & \text{otherwise} \quad \text{(state: interior)}, \end{cases} \tag{4}$$

and $\Delta F = F_{i+1} - F_i$.

This compact rule reproduces every case listed in the hint of the assignment:

- **Interior, no yielding:** the trial stays between the borders and $\Delta F = k_p \Delta x$.
- **Interior, yielding during the step:** the trial passes a border; the force is placed on the border, which is the same as moving with k_p up to the border and with k_s afterwards.
- **On the top border, $\Delta x > 0$:** $F^{\text{trial}} = F_{\text{top}}(x_i) + k_p \Delta x > F_{\text{top}}(x_{i+1})$, so $\Delta F = k_s \Delta x$ and the state stays `top`.
- **On the top border, $\Delta x < 0$:** the element unloads with k_p into the interior; if the unloading is large enough to reach the bottom border, the step ends on the bottom border.
- The bottom border is handled symmetrically.

A zero increment returns $\Delta F = 0$ and leaves the state unchanged, which avoids a state decision based on round-off when the point lies exactly on a border.

The element requires $k_s < k_p$ (otherwise the borders would be steeper than the elastic branch and the rule above would not hold); the function raises an error if this is violated.

2.2 Algorithm

Algorithm 1 Bilinear element, one displacement step

Require: $d_y, f_y, k_s, x_i, F_i, \Delta x$, state
1: $k_p \leftarrow f_y / d_y, \quad q_y \leftarrow f_y - k_s d_y$
2: **if** $\Delta x = 0$ **then return** 0, state
3: **end if**
4: $x_{i+1} \leftarrow x_i + \Delta x$
5: $F_{\text{top}} \leftarrow q_y + k_s x_{i+1}, \quad F_{\text{bot}} \leftarrow -q_y + k_s x_{i+1}$
6: $F^{\text{trial}} \leftarrow F_i + k_p \Delta x$
7: **if** $F^{\text{trial}} \geq F_{\text{top}}$ **then return** $F_{\text{top}} - F_i$, `top`
8: **else if** $F^{\text{trial}} \leq F_{\text{bot}}$ **then return** $F_{\text{bot}} - F_i$, `bottom`
9: **else return** $F^{\text{trial}} - F_i$, `interior`
10: **end if**

2.3 Program structure

The script `bilinear_element.py` (Appendix A) contains

- `bilinear(...)`: the element function described above;
- `run_history(x, dy, fy, k2)`: drives the element through a displacement history, starting from $F = 0$ in the interior region, and accumulates $F_{i+1} = F_i + \Delta F$;
- `save_history_plot`, `save_hysteresis_plot`: the figures;
- `main()`: the test case of the assignment and the printed results table.

3 Use of AI

As required in Part C, the code was developed with an AI coding assistant (Claude, Anthropic, used through Claude Code). The assignment text was given to the assistant, which wrote the element function, the driver, the plotting functions and a separate verification script, ran them and checked the figures. The formulation in Section 2 and the results in Section 4 were reviewed by the student. The verification described in Section 4.3 was used to confirm that the AI-generated code is correct rather than relying on the code by inspection.

4 Test of the bilinear element

4.1 Input

The test parameters are listed in Table 1. The input displacement history is

$$x = [\,0,\ 0.5,\ 0.75,\ 1.25,\ 1.5,\ 1.0,\ 1.75,\ 0.75,\ 0.25,\ 0.5,\ 0.0,\ -0.5,\\ -0.75,\ -1.5,\ -1.0,\ -1.75,\ -2.0,\ -2.5,\ -2.25,\ -1.75,\ -1.0,\ -0.25,\ 0,\ 1.0\,].$$

No time step is given, so the time axis of the time histories is the step number.

Table 1: Input and derived parameters.

Parameter	Symbol	Value
Yield displacement	d_y	1
Yield force	f_y	5
Secondary stiffness	k_s	0.25
Primary stiffness	$k_p = f_y/d_y$	5
Characteristic strength	$q_y = f_y - k_s d_y$	4.75

4.2 Results

The computed forces and states are listed in Table 2. The displacement and force time histories are shown in Figs. 1 and 2, and the hysteresis loop with the two borders in Fig. 3.

Table 2: Force and state at each step.

Step	x	F	State	Step	x	F	State
0	0.00	0.0000	interior	12	−0.75	−4.9375	bottom
1	0.50	2.5000	interior	13	−1.50	−5.1250	bottom
2	0.75	3.7500	interior	14	−1.00	−2.6250	interior
3	1.25	5.0625	top	15	−1.75	−5.1875	bottom
4	1.50	5.1250	top	16	−2.00	−5.2500	bottom
5	1.00	2.6250	interior	17	−2.50	−5.3750	bottom
6	1.75	5.1875	top	18	−2.25	−4.1250	interior
7	0.75	0.1875	interior	19	−1.75	−1.6250	interior
8	0.25	−2.3125	interior	20	−1.00	2.1250	interior
9	0.50	−1.0625	interior	21	−0.25	4.6875	top
10	0.00	−3.5625	interior	22	0.00	4.7500	top
11	−0.50	−4.8750	bottom	23	1.00	5.0000	top

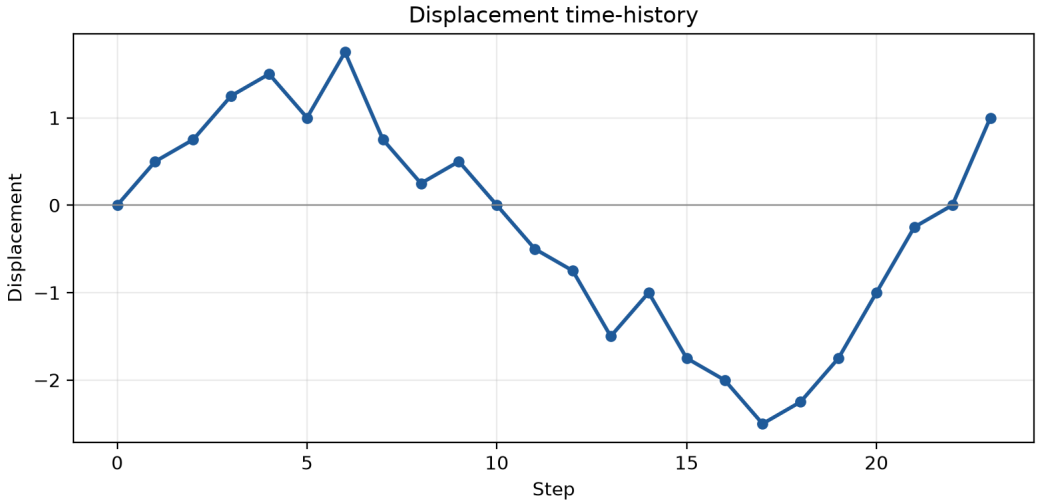


Figure 1: Displacement time history.

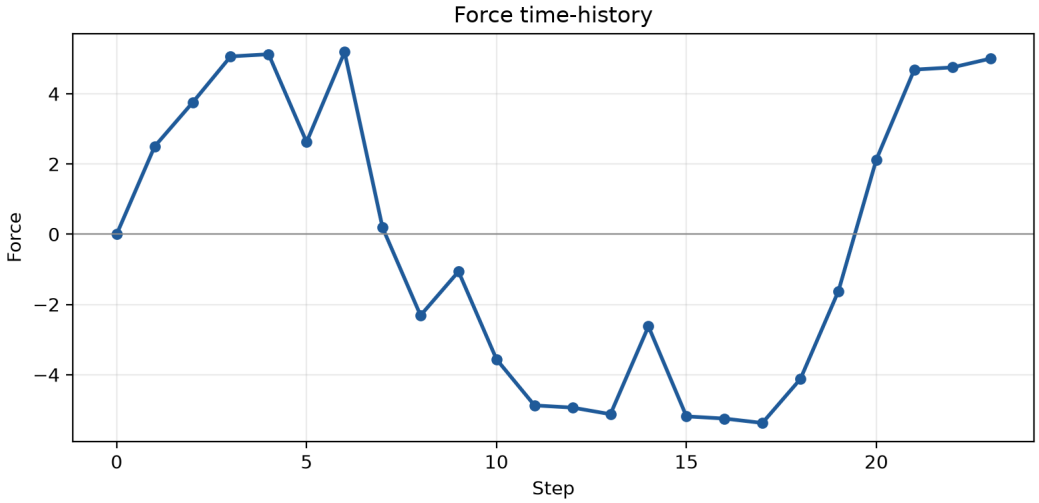


Figure 2: Force time history.

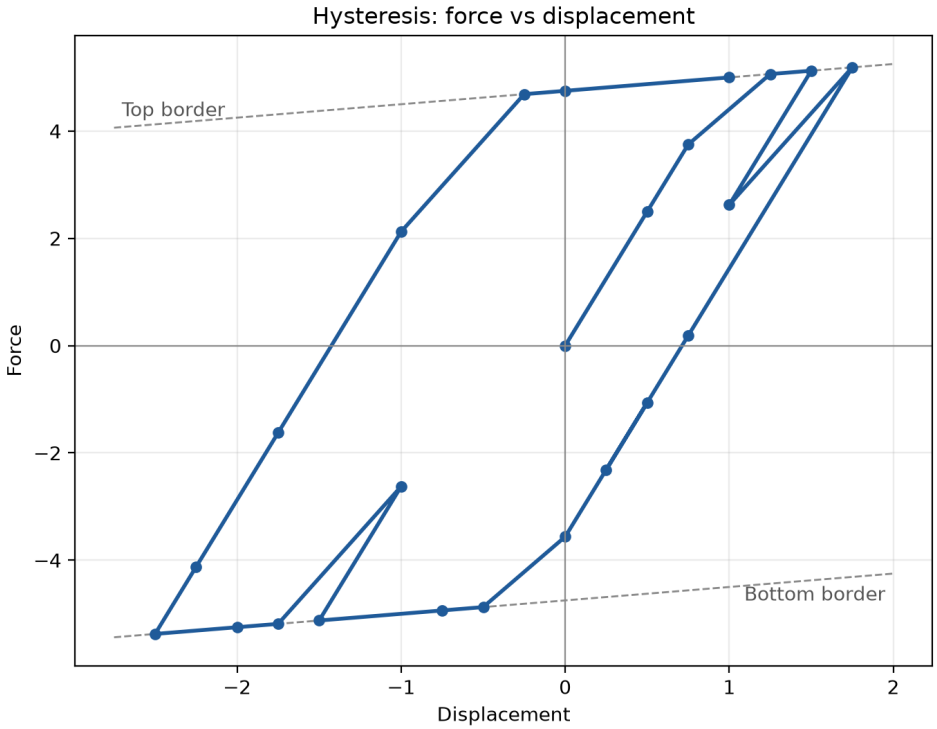


Figure 3: Hysteresis: force vs. displacement. Dashed lines are the top and bottom borders, Eq. (2).

Discussion. From step 0 to step 2 the element is elastic with slope $k_p = 5$. Yielding occurs during step 3 (at $x = 1$, $F = 5$) and the response then follows the top border with slope $k_s = 0.25$. Unloading in step 5 and step 7 follows the elastic slope; in step 7 the displacement decreases by 1.0, which changes the force by -5 and leaves the point inside the interior region because the elastic range between the borders is $2q_y = 9.5$ in force. Reloading in step 6 reaches the top border again. In the negative direction the element yields on the bottom border during step 11 and reaches its minimum force $F = -5.375$ at the peak displacement $x = -2.5$. Reloading from step 18 to step 21 crosses the whole interior region with slope k_p and reaches the top border during step 21. The hysteresis loop of Fig. 3 is a parallelogram whose elastic branches are parallel to each other and whose plastic branches lie on the two borders, which is the expected bilinear behaviour.

4.3 Verification

The AI-generated code was checked in three independent ways:

1. **Hand calculation.** Steps 1–11 were computed by hand. For example, in step 3 the trial force is $3.75 + 5(0.5) = 6.25 > F_{\text{top}}(1.25) = 4.75 + 0.25(1.25) = 5.0625$, so $F = 5.0625$; in step 11 the trial force is $-3.5625 - 5(0.5) = -6.0625 < F_{\text{bot}}(-0.5) = -4.875$, so $F = -4.875$. All hand values agree exactly with Table 2.

2. **Independent path tracing.** Each step was divided into 20 000 sub-steps and the force was integrated with the incremental stiffness (k_s when loading outward on a border, k_p otherwise). The largest difference from the bilinear function is about 8×10^{-12}.

3. **Consistency checks.** At every step the force lies between the two borders, and every step reported as `top` or `bottom` lies exactly on that border.

5 Conclusions

- A bilinear element was implemented in Python with the signature required by the assignment, using an elastic trial and a return to the crossed border.
- The element was tested with the given parameters ($d_y = 1$, $f_y = 5$, $k_s = 0.25$) and displacement history. The maximum and minimum forces are 5.1875 (at $x = 1.75$) and -5.375 (at $x = -2.5$).
- The results agree with hand calculations and with an independent fine-step solution, so the AI-generated code is considered correct for this test.

A Python code

```
"""
DEP 521E - Homework 2, Part C: Bilinear element model and testing (coded with AI).

Run: python bilinear_element.py
Output: displacement_history.png, force_history.png, hysteresis.png and a results table.

Time is expressed as the step index because the assignment gives no time step.
"""

from pathlib import Path

import matplotlib.pyplot as plt

INTERIOR = "interior"
TOP = "top"
BOTTOM = "bottom"

LINE_COLOR = "#1f5a99"
BORDER_COLOR = "#8a8a8a"


def bilinear(dy, fy, k2, x_ini, f_ini, dx, param_a=INTERIOR):
    """Return the force increment of a bilinear element for one displacement step.

    dy, fy : yield displacement and yield force
    k2 : secondary stiffness (must be smaller than fy / dy)
    x_ini : displacement at the start of the step
    f_ini : force at the start of the step
    dx : displacement increment
    param_a : state at the start of the step (INTERIOR, TOP or BOTTOM)

    Returns (df, param_a), with param_a updated to the state at the end of the step.
    """
    kp = fy / dy
    if k2 >= kp:
        raise ValueError("k2 must be smaller than the primary stiffness fy / dy")

    if dx == 0:
        return 0.0, param_a

    qy = fy - k2 * dy
    x_end = x_ini + dx
    f_top = qy + k2 * x_end
    f_bottom = -qy + k2 * x_end

    # Elastic trial with kp; a trial beyond a border means the step ends on that border.
    f_trial = f_ini + kp * dx
    if f_trial >= f_top:
```

```
        return f_top - f_ini, TOP
    if f_trial <= f_bottom:
        return f_bottom - f_ini, BOTTOM
    return f_trial - f_ini, INTERIOR


def run_history(x, dy, fy, k2):
    """Drive the element through displacement history x, starting from zero force."""
    forces = [0.0]
    states = [INTERIOR]
    for x_ini, x_end in zip(x[:-1], x[1:]):
        df, state = bilinear(dy, fy, k2, x_ini, forces[-1], x_end - x_ini, states[-1])
        forces.append(forces[-1] + df)
        states.append(state)
    return forces, states


def save_history_plot(y, ylabel, title, path):
    steps = range(len(y))
    fig, ax = plt.subplots(figsize=(8, 4))
    ax.plot(steps, y, color=LINE_COLOR, linewidth=2, marker="o", markersize=5)
    ax.axhline(0, color=BORDER_COLOR, linewidth=0.8)
    ax.set_xlabel("Step")
    ax.set_ylabel(ylabel)
    ax.set_title(title)
    ax.grid(alpha=0.25)
    fig.tight_layout()
    fig.savefig(path, dpi=200)
    plt.close(fig)


def save_hysteresis_plot(x, f, dy, fy, k2, path):
    qy = fy - k2 * dy
    x_range = [min(x) - 0.25, max(x) + 0.25]

    fig, ax = plt.subplots(figsize=(7, 5.5))
    ax.plot(x_range, [qy + k2 * v for v in x_range], color=BORDER_COLOR, linestyle="--", linewidth
        =1)
    ax.plot(x_range, [-qy + k2 * v for v in x_range], color=BORDER_COLOR, linestyle="--", linewidth
        =1)
    ax.plot(x, f, color=LINE_COLOR, linewidth=2, marker="o", markersize=5)
    ax.annotate("Top border", (x_range[0], qy + k2 * x_range[0]), xytext=(4, 6),
                textcoords="offset points", color="#555555")
    ax.annotate("Bottom border", (x_range[1], -qy + k2 * x_range[1]), xytext=(-4, -14),
                textcoords="offset points", ha="right", color="#555555")
    ax.axhline(0, color=BORDER_COLOR, linewidth=0.8)
    ax.axvline(0, color=BORDER_COLOR, linewidth=0.8)
    ax.set_xlabel("Displacement")
    ax.set_ylabel("Force")
    ax.set_title("Hysteresis: force vs displacement")
    ax.grid(alpha=0.25)
    fig.tight_layout()
    fig.savefig(path, dpi=200)
    plt.close(fig)


def main():
    dy = 1.0
    fy = 5.0
    k2 = 0.25
    x = [0, 0.5, 0.75, 1.25, 1.5, 1.0, 1.75, 0.75, 0.25, 0.5, 0.0, -0.5,
         -0.75, -1.5, -1.0, -1.75, -2.0, -2.5, -2.25, -1.75, -1.0, -0.25, 0, 1.0]

    f, states = run_history(x, dy, fy, k2)
```

```
111
112     print(f"kp = {fy / dy:g}, ks = {k2:g}, qy = {fy - k2 * dy:g}")
113     print(f"{'Step':>4} {'x':>7} {'F':>10} State")
114     for i, (xi, fi, state) in enumerate(zip(x, f, states)):
115         print(f"{i:>4} {xi:>7.2f} {fi:>10.4f} {state}")
116
117     out = Path(__file__).parent
118     save_history_plot(x, "Displacement", "Displacement time-history", out / "displacement_history.
            png")
119     save_history_plot(f, "Force", "Force time-history", out / "force_history.png")
120     save_hysteresis_plot(x, f, dy, fy, k2, out / "hysteresis.png")
121
122
123 if __name__ == "__main__":
124     main()
```